



Module 3

Natural Language Representation

CS 584

Schaefer School of Engineering

Justo Karell



Overview

- 1 **Core Definitions**
- 2 **Formal Languages**
- 3 **Natural vs Formal Languages**
- 4 **Language Data**
- 5 **Counting and Statistics**
- 6 **Example**

The Need for Definition

Before processing language, we need definitions.

Questions:

- What's a word, token, type?
- What's a phrase, sentence, corpus?
- How do we count language?

These definitions enable systematic analysis.



What is NLP

Processing natural language using formal methods

Definition: Word

A **word** is a space-separated unit.

Example: "The cat sat on the mat"

Words: The, cat, sat, on, the, mat

This definition is approximate.

"New York" = two words

"don't" = one word

Lecture 2 refines this.

Definition: Token

A **token** is a semantically valuable substring (e.g., a word, or subword)

Example: "the cat sat on the mat"

Tokens:

- Token 1: the
- Token 2: cat
- Token 3: sat
- Token 4: on
- Token 5: the
- Token 6: mat

Total: $N = 6$ tokens

Count every occurrence, including repeats.

Definition: Type

A **type** is each unique, singular word.

Example: "the cat sat on the mat"

Types: {the, cat, sat, on, mat}

Total: $|V| = 5$ types

Count each word once.

"the" appears twice = 1 type

Token vs Type

- **Types are unique**
- **Types are usually not subwords**
- Usually $|V| \leq N$

Each type appears at least once. If every word unique: $|V| = N$ If words repeat: $|V| < N$

Notation:

- N = total tokens
- $|V|$ = vocabulary size

Definition: Phrase

A **phrase** is a grammatical unit.

Example: "the cat sat on the mat"

Phrases:

- "the cat" (noun phrase)
- "on the mat" (prepositional phrase)
- "the mat" (noun phrase)

Internal structure, not complete sentences.

Definition: Sentence

A **sentence** is a complete unit.

Properties:

- Subject and predicate
- Complete thought
- Terminal punctuation (. ! ?)

Example: "The cat sat on the mat."

6 tokens, 5 types

Definition: Corpus

A **corpus** is computer-readable text.

Plural: **corpora**

Properties:

- Multiple sentences
- Large scale (thousands to billions)
- Represents language variety
- Trains NLP systems

Example:

The cat sat on the mat. The dog ran in the park.

2 sentences, 13 tokens, 11 types

Visual Hierarchy



Counting dimensions:

- Token = count each occurrence
- Type = count each unique

Definitions enable systematic language processing.

Why Formal Languages?

Need mathematical framework for computation.

Formal language theory provides:

- Precise notation
- Systematic methods
- Testing procedures

Enables computational language handling.

Definition: Alphabet

An **alphabet** Σ is finite symbols.

Examples:

- $\Sigma = \{a, b\}$
- $\Sigma = \{A, B, \dots, Z\}$
- $\Sigma = \{0, 1\}$

Σ is finite and fixed.

Definition: String

A **string** is a symbol sequence.

Examples with $\Sigma = \{a, b\}$:

- a (length 1)
- ab (length 2)
- $abba$ (length 4)
- ϵ (empty, length 0)

Σ^* = all possible strings

Includes ϵ , infinitely many strings

Definition: Formal Language

A **formal language** L is $L \subseteq \Sigma^*$

L is a string collection.

Can be infinite.

Question: Is $s \in L$?

This is **membership testing**.

Languages Are Defined By Rules

Languages specify valid strings.

Example: "even number of 1's"

$\Sigma = \{0, 1\}$ defines L

Membership test:

- 1 Count 1's
- 2 Even (including zero) $\rightarrow \in L$
- 3 Odd $\rightarrow \notin L$

Example: Membership Testing

$$\Sigma = \{0, 1\}$$

$$L = \{\text{even number of 1's}\}$$

For each string:

- 1 Count 1's
- 2 Check parity
- 3 Accept or reject

Example: Tests

$$L = \{\text{even 1's}\}$$

$$\epsilon: 0 \text{ (even)} \rightarrow \epsilon \in L$$

$$0: 0 \text{ (even)} \rightarrow 0 \in L$$

$$1: 1 \text{ (odd)} \rightarrow 1 \notin L$$

$$11: 2 \text{ (even)} \rightarrow 11 \in L$$

$$010: 1 \text{ (odd)} \rightarrow 010 \notin L$$

$$0110: 2 \text{ (even)} \rightarrow 0110 \in L$$

$$101101: 4 \text{ (even)} \rightarrow 101101 \in L$$

Membership Testing

We just performed membership testing.

Process:

- ① Language defined by rule
- ② Test strings against rule
- ③ Accept or reject

Foundation of formal language theory.
Later: Grammar rules test sentences.

Natural Language Processing

NLP: computational analysis and generation.

Challenge: ambiguous, variable, unbounded.

Bridges:

- Human communication
- Computer algorithms

Uses formal methods on natural language.

Comparing Language Types

	Formal	Natural
Alphabet	Fixed	Open
Rules	Precise	Fuzzy
Ambiguity	None	Everywhere
Membership	Decidable	Judgment
New symbols	Never	Constantly

Natural languages violate formal assumptions.
Different strategies required.

Challenge: Open Alphabet

Formal: Σ fixed forever

Natural: new words constantly

Examples:

- Emojis (,)
- Acronyms (LOL, FOMO)
- Names (companies, people, places)
- Technical terms (smartphone, cryptocurrency)

No finite alphabet captures natural language.

Challenge: Fuzzy Rules

Formal: precise, mechanical rules

Natural: exceptions and gradations

Example: "Colorless green ideas sleep furiously"

- Grammatically correct
- Semantically meaningless
- Which rule matters?

Simple membership testing fails.

Challenge: Ambiguity

Three types:

- Lexical: same word, different meanings
 - "I need to bank the money"
 - "The plane flew along the river bank"
- Syntactic: same words, different structure
 - "I saw the man with the telescope"
- Semantic: context-dependent
 - "The chicken is ready to eat"



Why These Challenges Matter

- Formal tools don't directly apply.
 - Can't enumerate valid sentences.
 - Can't predict word combinations.
 - Can't write rules for ambiguity.
- Solution: learn from data → requires corpora / language data

What a Corpus Looks Like

In Python, a corpus is text:

```
corpus_text = """  
The_cat_sat_on_the_mat.  
The_dog_walked_in_the_park.  
"""
```

Text in memory.

Loaded from file.

Any length (memory permitting).

Corpus	Size (words)
Brown corpus	1,000,000
Switchboard	2,400,000
Wikipedia	Billions
Google n-grams	1 trillion

Counting "words" per earlier definition.
More tokens = more patterns.



From Definitions to Statistics

Definitions:

- Tokens (count all)
- Types (count unique)

Apply to compute corpus statistics.
Reveals language patterns.

Example: Counts

Sentence:

They picnicked by the pool then lay back on the grass and looked at the stars

All tokens: They, picnicked, by, the, pool, then, lay, back, on, the, grass, and, looked, at, the, stars

$N = 16$ tokens

Unique types: {They, picnicked, by, the, pool, then, lay, back, on, grass, and, looked, at, stars}

$|V| = 14$ types

"the" appears 3 times: 3 tokens, 1 type



Python for Counting

```
tokens = ["the", "cat", "sat", "on", "the", "mat"]
```

```
# Count tokens
```

```
N = len(tokens)
```

```
print(f"Tokens:_{N}") # 6
```

```
# Count types
```

```
vocab = set(tokens)
```

```
V = len(vocab)
```

```
print(f"Types:_{V}") # 5
```

list for tokens (allows duplicates)

set for types (removes duplicates)



Corpus Statistics

Corpus	Types ($ V $)	Tokens (N)
Shakespeare	31,000	884,000
Brown	38,000	1,000,000
Switchboard	20,000	2,400,000
COCA	2,000,000	440,000,000

As N grows, $|V|$ grows.

$|V|$ grows slower than N .

What's the relationship?

Vocabulary Growth: Heaps' Law

Vocabulary grows sublinearly.

Heaps' Law (Herdan's Law):

$$|V| = kN^\beta$$

where:

- k, β positive constants
- $0 < \beta < 1$ (typically 0.4 to 0.6)
- Common: $\beta \approx 0.5$

Approximation: $|V| \approx \sqrt{N}$

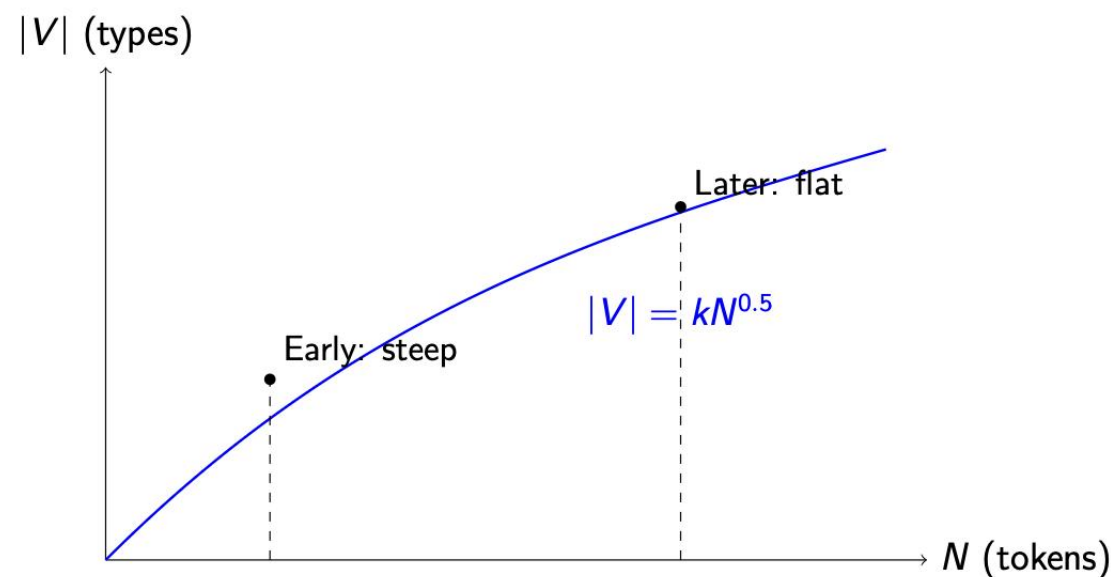
Heaps' Law: Interpretation

Diminishing returns:

- First 1,000 words: 500 unique
- Next 1,000 words: 100 new
- Growth slows

But never stops:

- $N \rightarrow \infty \Rightarrow |V| \rightarrow \infty$
- Always encounter new words



Heaps' Law: Implication

No vocabulary is ever complete.

Systems encounter unknown words.

Must design for:

- Unseen vocabulary
- Open-ended lists
- Continuous learning

Motivates later techniques.



Thank you!

Date